Workforce Forecasting System

Complete Code Documentation

A comprehensive guide to the workforce forecasting application

Workforce Forecasting System - Complete Code Documentation

Table of Contents

_

Project Overview

The Workforce Forecasting System is a comprehensive machine learning application designed to predict workforce demand across different departments. The system consists of three main components:

• **Backend**: Java Spring Boot application for data management and API services

• **Frontend**: Vue.js application for user interface and visualization

• **Python Service**: Machine learning models for training and prediction

Technology Stack

Backend:

• Java 17+

• Spring Boot

• Spring Data JPA

• PostgreSQL/MySQL database

• RESTful APIs

Frontend:

• Vue.js 3

• TypeScript

• PrimeVue components

• Chart.js for visualizations

• Axios for HTTP requests

• Tailwind CSS

Python ML Service:

• FastAPI

• TensorFlow/Keras

• scikit-learn

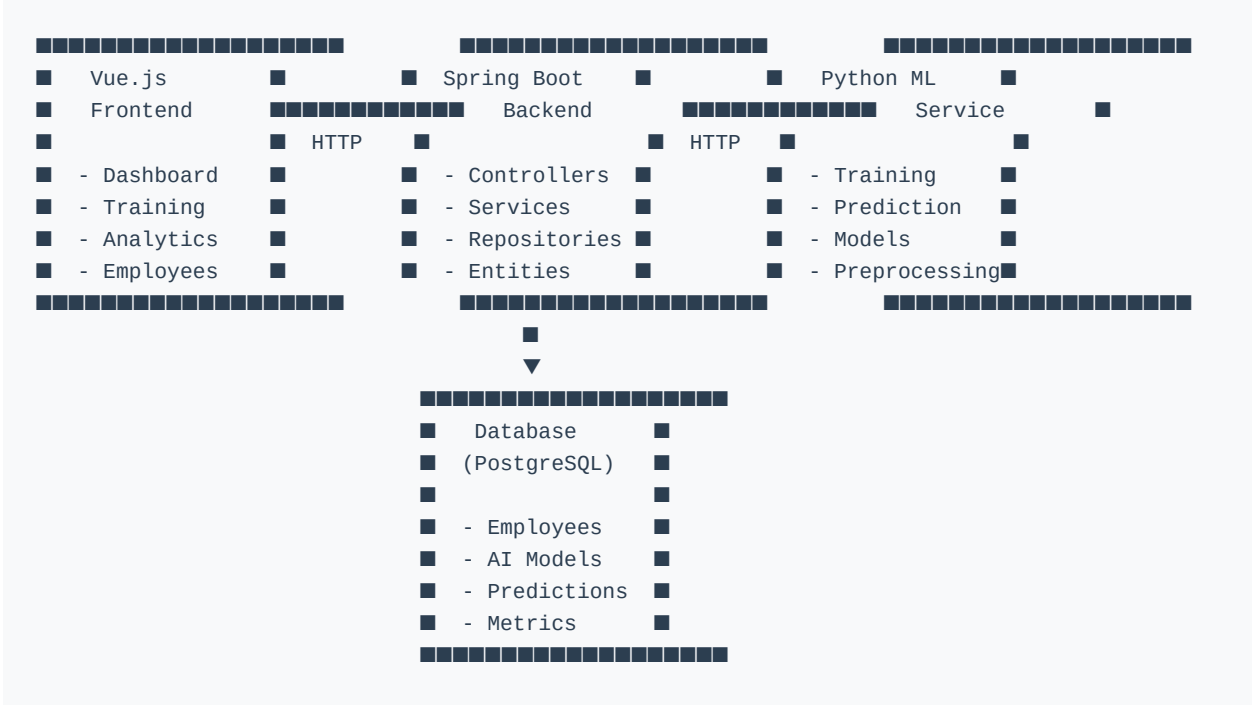
• XGBoost

• Pandas

• NumPy

_

System Architecture



_

Backend Implementation (Java Spring Boot)

Main Application Class

File: `WorkforceApplication.java`

```
package com.boostphysioclinic.workforceapplication;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class WorkForceApplication {
    public static void main(String[] args) {
        SpringApplication.run(WorkForceApplication.class, args);
    }
}
```

Explanation:

• Main entry point for the Spring Boot application

• `@SpringBootApplication` annotation enables auto-configuration and component scanning

• Starts the embedded web server (Tomcat) and initializes the Spring context

Controllers

DashboardController

File: `DashboardController.java`

Purpose: Provides dashboard metrics and visualization data for the frontend.

Key Endpoints:

1. **GET** `/api/dashboard`

• Returns comprehensive dashboard data including metrics and charts

• Fetches latest trained model information from database

• Calculates real-time metrics from prediction data

• Returns line chart data (historical vs predicted demand)

• Returns bar chart data (department performance)

2. **GET** `/api/dashboard/predictions`

• Returns prediction records from CSV files

• Used for populating prediction tables

3. **GET** `/api/dashboard/overview`

• Alias for the main dashboard endpoint

Key Functionality:

```
@GetMapping
public ResponseEntity<Map<String, Object>> getDashboard() {
    // Fetch latest trained model
    Optional<AIModel> latestModel = aiModelRepository.findFirstByOrderByLastTrainedDesc();

    // Get predictions
    List<PredictionRecord> predictions = predictionCsvService.getPredictions();

    // Calculate metrics
    double totalPredicted = predictions.stream()
        .mapToDouble(PredictionRecord::getPredictedDemand)
        .sum();

    // Prepare chart data
    Map<String, Object> lineChartData = new HashMap<>();
    lineChartData.put("labels", labels);
    lineChartData.put("historical", historicalData);
    lineChartData.put("predicted", predictedData);

    return ResponseEntity.ok(response);
}
```

TrainingController

File: `TrainingController.java`

Purpose: Handles model training requests and manages training results.

Key Endpoints:

1. **POST** `/api/train`

• Accepts CSV file and selected algorithms

• Forwards training request to Python ML service

• Saves training results to database

• Returns training response with metrics

2. **GET** `/api/train/cleaned-dataset`

• Downloads the preprocessed dataset used for training

3. **GET** `/api/train/latest-model`

• Returns the most recently trained model information

4. **GET** `/api/train/model-comparisons`

• Returns comparison data for all trained models

Key Functionality:

```
@PostMapping
public ResponseEntity<String> trainModel(
    @RequestParam("file") MultipartFile file,
    @RequestParam("algorithms") List<String> algorithms
) throws Exception {
    return ResponseEntity.ok(trainingService.train(file, algorithms));
}
```

Services

TrainingService

File: `TrainingService.java`

Purpose: Orchestrates model training by communicating with Python ML service.

Key Methods:

1. `train(MultipartFile file, List algorithms)`

• Converts uploaded file to ByteArrayResource

• Constructs multipart request with file and algorithms

• Sends POST request to Python training endpoint

• Parses response and saves results to database

• Returns training response to frontend

2. `saveTrainingResults(String responseBody, String fileName, List algorithms)`

• Parses JSON response from Python service

• Extracts best model and metrics

• Saves AIModel entity with performance metrics

• Saves ModelComparison entities for all trained models

Key Code:

```
public String train(MultipartFile file, List<String> algorithms) throws Exception {
    RestTemplate restTemplate = new RestTemplate();

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.MULTIPART_FORM_DATA);

    ByteArrayResource resource = new ByteArrayResource(file.getBytes()) {
        @Override
        public String getFilename() {
            return file.getOriginalFilename();
        }
    };

    MultiValueMap<String, Object> body = new LinkedMultiValueMap<>();
    body.add("file", resource);
    body.add("algorithms", String.join(",", algorithms));

    ResponseEntity<String> response = restTemplate.postForEntity(
        pythonUrl, request, String.class
    );

    saveTrainingResults(response.getBody(), file.getOriginalFilename(), algorithms);
    return response.getBody();
}
```

Repositories

The backend uses Spring Data JPA repositories for database operations:

• **AIModelRepository**: Manages trained AI model metadata

• **ModelComparisonRepository**: Stores comparison results between models

• **EmployeeRepository**: Manages employee data

• **PredictionResultRepository**: Stores prediction results

• **DatasetRepository**: Manages training datasets

Entities

AIModel

Purpose: Represents a trained machine learning model with its performance metrics.

Key Fields:

• `name`: Model name

• `algorithm`: Algorithm type (Linear Regression, Random Forest, XGBoost, LSTM)

• `version`: Model version

• `status`: Training status (TRAINED, TRAINING, FAILED)

• `rmse`: Root Mean Square Error

• `mae`: Mean Absolute Error

• `mape`: Mean Absolute Percentage Error

• `rSquared`: R-squared score

• `lastTrained`: Timestamp of last training

ModelComparison

Purpose: Stores comparison metrics for different models trained in the same session.

Key Fields:

• `modelName`: Name of the model

• `algorithm`: Algorithm type

• `rmse`, `mae`, `mape`, `rSquared`: Performance metrics

• `status`: Model status (Best, Good)

• `createdAt`: Creation timestamp

_

Frontend Implementation (Vue.js)

Main Application Setup

File: `main.ts`

Purpose: Application entry point that configures Vue, plugins, and global components.

Key Configurations:

1. **PrimeVue Setup**

```
app.use(PrimeVue, {
  theme: {
    preset: Aura,
    options: {
      darkModeSelector: '.dark'
    }
  }
})
```

2. **Global Components Registration**

• Chart, Panel, Card, Button, Tag, Divider

• DataTable, Column, InputText, Select

• FileUpload, Toast, Dropdown, InputNumber

3. **Chart.js Registration**

```
ChartJS.register(
  LineController, LineElement, BarElement, BarController,
  CategoryScale, LinearScale, PointElement, RadialLinearScale,
  Filler, Tooltip, Legend
)
```

App Component

File: `App.vue`

Purpose: Main application layout with sidebar and top menu.

Key Features:

• Conditional rendering of sidebar and top bar based on route

• Responsive layout with flexbox

• Scrollable content area with custom scrollbar styling

• Toast notification integration

Layout Structure:

```
<template>
  <div class="app-wrapper">
    <Toast />
    <topMenuBarComponent v-if="showTopBar" />
    <div class="layout">
      <div class="sideBar" v-if="showSidebar">
        <sideBarComponent />
      </div>
      <div class="content">
        <router-view />
      </div>
    </div>
  </div>
</template>
```

API Client

File: `apiClient.ts`

Purpose: Configured Axios instance for HTTP requests with authentication.

Key Features:

1. **Base Configuration**

```
const api = axios.create({
  baseURL: '/api',
  timeout: 1200000, // 20 minutes for long operations
})
```

2. **JWT Token Interceptor**

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})
```

3. **Error Handling**

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token')
      window.location.href = '/login'
    }
    return Promise.reject(error)
  }
)
```

Dashboard Component

File: `dashboardComponent.vue`

Purpose: Main dashboard displaying workforce metrics, charts, and operational alerts.

Key Features:

1. **KPI Cards**

• Active Workforce

• Forecast Accuracy

• Shift Utilization

• Total Predictions

• Average Demand

• Model Status

• Model Name

• R^2 Score

• RMSE

2. **Line Chart**

• Historical vs Predicted demand over time

• Uses Chart.js with custom styling

• Auto-refreshes every 30 seconds

3. **Bar Chart**

• Department performance comparison

• Horizontal bar chart with target lines

• Color-coded performance indicators

4. **Operational Alerts**

• Real-time alerts with severity levels

• Critical, Warning, Success categories

• Time-stamped alert notifications

5. **Staffing Heatmap**

• Department-wise shift allocation

• Morning, Afternoon, Night shifts

• Total staffing per department

Key Methods:

```
const fetchDashboardMetrics = async () => {
  loading.value = true
  try {
    const response = await CLDashboardService.getDashboardData()

    if (response?.metrics) {
      const backendMetrics = response.metrics as Record<string, string>
      // Update KPI cards with backend data
      metrics.value = [
        {
          title: 'Active Workforce',
          value: backendMetrics['Average Demand'] || 'N/A',
          // ... other metrics
        }
      ]
    }

    // Update chart data
    if (response?.charts) {
      const charts = response.charts as Record<string, any>
      chartData.value = setChartData(
        charts.lineChart.labels,
        charts.lineChart.historical,
        charts.lineChart.predicted
      )
    }
  } catch (error) {
    console.error('Failed to fetch dashboard metrics:', error)
  } finally {
    loading.value = false
```

```
  }
}
```

Auto-refresh Mechanism:

```
onMounted(() => {
  fetchDashboardMetrics()
  // Set up real-time refresh every 30 seconds
  refreshInterval = window.setInterval(() => {
    fetchDashboardMetrics()
  }, 30000)
})
```

_

Python ML/Forecasting Service

Main Application

File: `serviceFast/main.py`

Purpose: FastAPI application entry point that registers API routers.

Code:

```
from fastapi import FastAPI
from serviceFast.controller.train_controller import router as train_router
from serviceFast.controller.prediction_controller.prediction_router import router as prediction_router

app = FastAPI()
app.include_router(train_router)
app.include_router(prediction_router)
```

Training Pipeline

File: `serviceFast/controller/train_pipeline.py`

Purpose: Orchestrates the complete training workflow from data loading to model saving.

Workflow:

1. Load and preprocess dataset
2. Split features and target
3. Train selected models
4. Evaluate models
5. Save best model and results
6. Return training metrics

Model Training

File: `training/model_trainer.py`

Purpose: Coordinates training of multiple machine learning models.

Key Method:

```
class ModelTrainer:
    def train_models(self, selected_models, X_train, X_test, y_train, y_test, lstm_df):
        results = []
        trained_models = {}

        # Linear Regression
        if "Linear Regression" in selected_models:
            lr_model = train_linear_regression(X_train, y_train)
            predictions = lr_model.predict(X_test)
            metrics = evaluate_model(y_test, predictions, "Linear Regression")
            results.append(metrics)
            trained_models["Linear Regression"] = {
                "type": "ml",
                "model": lr_model
            }

        # Random Forest
        if "Random Forest" in selected_models:
            rf_model = train_random_forest(X_train, y_train)
            predictions = rf_model.predict(X_test)
            metrics = evaluate_model(y_test, predictions, "Random Forest")
            results.append(metrics)
            trained_models["Random Forest"] = {
                "type": "ml",
                "model": rf_model
            }

        # XGBoost
        if "XGBoost" in selected_models:
            xgb_model = train_xgboost(X_train, y_train)
```

```
            predictions = xgb_model.predict(X_test)
            metrics = evaluate_model(y_test, predictions, "XGBoost")
            results.append(metrics)
            trained_models["XGBoost"] = {
                "type": "ml",
                "model": xgb_model
            }

        # LSTM (Deep Learning)
        if "LSTM" in selected_models:
            X_train_lstm, X_test_lstm, y_train_lstm, y_test_lstm, X_dashboard, target_scaler, test_departmen
            lstm_model, history = train_lstm(X_train_lstm, y_train_lstm)
            predictions = lstm_model.predict(X_test_lstm, verbose=0)
            predictions = target_scaler.inverse_transform(predictions)
            y_actual = target_scaler.inverse_transform(y_test_lstm.reshape(-1, 1))
            metrics = evaluate_model(y_actual.flatten(), predictions.flatten(), "LSTM")
            results.append(metrics)
            trained_models["LSTM"] = {
                "type": "lstm",
                "model": lstm_model,
                "dashboard_data": X_dashboard,
                "target_scaler": target_scaler
            }

        return results, trained_models, X_dashboard, target_scaler
```

LSTM Model

File: `models/lstm_model.py`

Purpose: Implements Long Short-Term Memory neural network for time series forecasting.

Architecture:

```
model = Sequential([
    Input(shape=(X_train.shape[1], X_train.shape[2])),

    # LSTM Block 1
    LSTM(128, return_sequences=True),
    BatchNormalization(),
    Dropout(0.30),

    # LSTM Block 2
    LSTM(64, return_sequences=False),
    BatchNormalization(),
    Dropout(0.30),

    # Dense Layers
    Dense(32, activation="relu"),
    Dropout(0.20),
    Dense(16, activation="relu"),
    Dense(1)
])
```

Training Configuration:

• Optimizer: Adam (learning rate=0.001)

• Loss: Mean Squared Error (MSE)

• Metrics: Root Mean Square Error (RMSE)

• Epochs: 150

• Batch Size: 64

• Validation Split: 20%

• Callbacks: Early Stopping, Learning Rate Reduction

Callbacks:

```
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=12,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=5,
    min_lr=1e-5,
    verbose=1
)
```

Random Forest Model

File: `models/random_forest_model.py`

Purpose: Implements Random Forest regressor with hyperparameter tuning.

Hyperparameter Grid:

```
param_grid = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [5, 10, 15, 20, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2"]
}
```

Optimization:

• Uses RandomizedSearchCV for efficient hyperparameter search

• 10 iterations with 3-fold cross-validation

• Scoring: Negative Root Mean Square Error

• Parallel processing with n_jobs=-1

Data Preprocessing

File: `preprocessing/preprocessing.py`

Purpose: Prepares raw workforce data for machine learning models.

Key Steps:

1. **Data Loading**

```
def load_dataset(path):
    df = pd.read_csv(path)
    return df
```

2. **Data Cleaning**

```
def preprocess_dataset(df):
    # Remove duplicates
    df.drop_duplicates(inplace=True)

    # Convert date
    df["AttendanceDate"] = pd.to_datetime(df["AttendanceDate"])

    # Sort by department and date
    df = df.sort_values(["Department", "AttendanceDate"])
```

3. **Feature Engineering**

```
# Calendar Features
df["Year"] = df["AttendanceDate"].dt.year
df["Quarter"] = df["AttendanceDate"].dt.quarter
df["Month"] = df["AttendanceDate"].dt.month
df["WeekOfYear"] = df["AttendanceDate"].dt.isocalendar().week.astype(int)
df["DayOfWeek"] = df["AttendanceDate"].dt.dayofweek
df["Weekend"] = (df["DayOfWeek"] >= 5).astype(int)
df["IsMonthStart"] = df["AttendanceDate"].dt.is_month_start.astype(int)
df["IsMonthEnd"] = df["AttendanceDate"].dt.is_month_end.astype(int)

# Historical Features
dept = df.groupby("Department")
df["PreviousDayDemand"] = dept["WorkforceDemand"].shift(1)
df["Previous3DayAverage"] = dept["WorkforceDemand"].transform(
    lambda x: x.shift(1).rolling(3, min_periods=1).mean()
)
df["Previous7DayAverage"] = dept["WorkforceDemand"].transform(
    lambda x: x.shift(1).rolling(7, min_periods=1).mean()
)

# Forecast Target
df["TargetDemand"] = dept["WorkforceDemand"].shift(-1)
```

4. **Data Encoding**

```
def encode_dataset(df):
    categorical_columns = df.select_dtypes(include=["object"]).columns.tolist()
    if "AttendanceDate" in categorical_columns:
        categorical_columns.remove("AttendanceDate")

    if len(categorical_columns) > 0:
        df = pd.get_dummies(df, columns=categorical_columns, drop_first=True)
    return df
```

5. **Train-Test Split**

```
def split_train_test(X, y):
    split_index = int(len(X) * 0.80)
    X_train = X.iloc[:split_index].copy()
    X_test = X.iloc[split_index:].copy()
    y_train = y.iloc[:split_index].copy()
    y_test = y.iloc[split_index:].copy()
    return X_train, X_test, y_train, y_test
```

Model Evaluation

File: `evaluation/evaluation.py`

Purpose: Evaluates model performance using multiple metrics.

Metrics Calculated:

```
def evaluate_model(y_true, predictions, model_name):
    # Root Mean Square Error
    rmse = np.sqrt(mean_squared_error(y_true, predictions))

    # Mean Absolute Error
    mae = mean_absolute_error(y_true, predictions)

    # Mean Absolute Percentage Error
    epsilon = 1e-8
    mape = np.mean(np.abs((y_true - predictions) / (y_true + epsilon))) * 100

    # R-squared Score
    r2 = r2_score(y_true, predictions)

    return {
        "Model": model_name,
        "RMSE": round(rmse, 4),
        "MAE": round(mae, 4),
        "MAPE": round(mape, 2),
        "R2": round(r2, 4)
    }
```

Prediction Service

File: `serviceFast/model_service/prediction_service.py`

Purpose: Handles prediction requests using trained models.

Key Functionality:

```
class PredictionService:
    def __init__(self):
        # Load trained model
        self.model = joblib.load(model_path)

        # Load training feature columns
        with open(feature_path, "r") as file:
            self.training_columns = json.load(file)

        # Load model info
        with open(info_path, "r") as file:
            self.model_info = json.load(file)

    def predict(self, dataframe):
        # Add missing columns with zeros
        missing_columns = [col for col in self.training_columns if col not in dataframe.columns]
        if missing_columns:
            missing_df = pd.DataFrame(0, index=dataframe.index, columns=missing_columns)
            dataframe = pd.concat([dataframe, missing_df], axis=1)

        # Keep only training columns
        dataframe = dataframe[self.training_columns]

        # Make predictions
        predictions = self.model.predict(dataframe)

        # Format results
        results = []
        for i, pred in enumerate(predictions):
            result = {
```

```
                "attendanceDate": dataframe.iloc[i].get("AttendanceDate", ""),
                "department": dataframe.iloc[i].get("Department", ""),
                "actualDemand": dataframe.iloc[i].get("WorkforceDemand", None),
                "predictedDemand": float(pred)
            }
            results.append(result)

        return {
            "model": self.model_info["Model"],
            "total_records": len(predictions),
            "predictions": predictions.tolist(),
            "results": results
        }
```

Prediction Controller

File:
`serviceFast/controller/prediction_controller/prediction_router.py`

Purpose: API endpoint for handling prediction requests.

Endpoint:

```
@router.post("")
async def predict(file: UploadFile = File(...)):
    try:
        # Read uploaded CSV
        dataframe = pd.read_csv(file.file)

        # Apply preprocessing
        prediction_df = prepare_prediction_data(dataframe)

        # Predict
        result = prediction_service.predict(prediction_df)

        return result
    except Exception:
        traceback.print_exc()
        raise HTTPException(
            status_code=500,
            detail="Prediction failed. Check server logs."
        )
```

_

Data Flow and Integration

Training Workflow

1. **User uploads CSV file** via Frontend UI

2. **Frontend sends POST request** to `/api/train` with file and selected algorithms

3. **Backend TrainingService** forwards request to Python ML service

4. **Python service** processes the file:

• Loads and preprocesses data

• Trains selected models

• Evaluates performance

• Saves best model

5. **Python returns** training results with metrics

6. **Backend saves** results to database (AIModel, ModelComparison tables)

7. **Backend returns** response to Frontend

8. **Frontend updates** UI with training results

Prediction Workflow

1. **User uploads prediction CSV** via Frontend

2. **Frontend sends POST request** to Python prediction endpoint

3. **Python service**:

• Preprocesses prediction data

• Loads trained model

• Makes predictions

• Returns formatted results

4. **Results displayed** in Frontend dashboard

Dashboard Data Flow

1. **Frontend polls** `/api/dashboard` every 30 seconds

2. **Backend fetches**:

• Latest trained model from database

• Prediction records from CSV files

3. **Backend calculates**:

• Real-time metrics (accuracy, averages)

• Chart data (line chart, bar chart)

4. **Frontend updates**:

• KPI cards

• Charts

• Alerts panel

_

Key Algorithms and Models

Linear Regression

Purpose Baseline model for workforce demand prediction.

Characteristics:

• Simple interpretable model

• Fast training time

• Good for linear relationships

• Limited capture of complex patterns

Random Forest

Purpose: Ensemble learning method for improved accuracy.

Characteristics:

• Multiple decision trees

• Handles non-linear relationships

• Robust to overfitting

• Feature importance analysis

• Moderate training time

XGBoost

Purpose: Gradient boosting framework for high performance.

Characteristics:

• Advanced gradient boosting

• Handles missing values

• Regularization to prevent overfitting

• High accuracy

• Faster training than Random Forest

LSTM (Long Short-Term Memory)

Purpose: Deep learning model for time series forecasting.

Characteristics:

• Captures temporal dependencies

• Handles sequential data

• Complex architecture with multiple layers

• Highest accuracy potential

• Longer training time

• Requires more data

Architecture Details:

• Two LSTM layers (128 and 64 units)

• Batch normalization for stability

• Dropout regularization (30% for LSTM, 20% for Dense)

• Dense layers for feature transformation

• Adam optimizer with learning rate scheduling

_

API Endpoints Reference

Backend API Endpoints

Dashboard Endpoints

- `GET /api/dashboard` - Get dashboard metrics and charts
- `GET /api/dashboard/predictions` - Get prediction records
- `GET /api/dashboard/overview` - Get dashboard overview

Training Endpoints

- `POST /api/train` - Train models with uploaded dataset
- Parameters: `file` (MultipartFile), `algorithms` (List)
- Returns: Training results with metrics
- `GET /api/train/cleaned-dataset` - Download preprocessed dataset
- `GET /api/train/latest-model` - Get latest trained model info
- `GET /api/train/model-comparisons` - Get model comparison data

Employee Endpoints

- `GET /api/employees` - Get all employees
- `POST /api/employees` - Create new employee
- `PUT /api/employees/{id}` - Update employee
- `DELETE /api/employees/{id}` - Delete employee

Analytics Endpoints

- `GET /api/analytics` - Get analytics data
- `GET /api/analytics/performance` - Get performance metrics

Authentication Endpoints

- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `POST /api/auth/register` - User registration

Python ML Service Endpoints

Training Endpoints

- `POST /train` - Train ML models
- Parameters: `file` (UploadFile), `algorithms` (comma-separated string)
- Returns: JSON with best model, metrics, and comparison data

Prediction Endpoints

- `POST /predict` - Make predictions
- Parameters: `file` (UploadFile with prediction data)
- Returns: JSON with predictions and metadata

_

Database Schema

AIModel Table

Column	Type	Description
id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type

Column	Type	Description
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error

id	BIGINT	Primary key
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error

version	VARCHAR	Model version
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

last_trained	TIMESTAMP	Last training timestamp

ModelComparison Table

Column	Type	Description
id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score

id	BIGINT	Primary key
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

model_name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
status	VARCHAR	Model status
created_at	TIMESTAMP	Creation timestamp

algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error

algorithm	VARCHAR	Algorithm type
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

status	VARCHAR	Model status
created_at	TIMESTAMP	Creation timestamp

created_at	TIMESTAMP	Creation timestamp

Employee Table

Column	Type	Description
id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version

id	BIGINT	Primary key
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

first_name	VARCHAR	Employee first name
last_name	VARCHAR	Employee last name
email	VARCHAR	Employee email
department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

last_name	VARCHAR	Employee last name
email	VARCHAR	Employee email
department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

email	VARCHAR	Employee email
department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

hire_date	DATE	Hire date
status	VARCHAR	Employment status

status	VARCHAR	Employment status

_

Configuration Files

Backend Configuration

application.properties

```
# Database Configuration
spring.datasource.url=jdbc:postgresql://localhost:5432/workforce_db
spring.datasource.username=postgres
spring.datasource.password=password
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

# Python Service Configuration
python.api.url=http://localhost:8000/train
python.download.url=http://localhost:8000/download/cleaned-dataset

# Server Configuration
server.port=8080
```

Frontend Configuration

vite.config.ts

```
export default defineConfig({
  plugins: [vue()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true
      }
    }
  }
})
```

Python Configuration

requirements.txt

```
fastapi==0.115.0
uvicorn==0.32.0
pydantic==2.9.2
python-multipart==0.0.12
numpy==1.26.4
pandas==2.1.0
scikit-learn
xgboost
tensorflow
joblib
```

_

Deployment Instructions

Backend Deployment

1. **Build the application:**

```
./gradlew build
```

2. **Run the application:**

```
java -jar build/libs/workForceApplication.jar
```

3. **Or run with Gradle:**

```
./gradlew bootRun
```

Frontend Deployment

1. **Install dependencies:**

```
npm install
```

2. **Development server:**

```
npm run dev
```

3. **Production build:**

```
npm run build
```

4. **Preview production build:**

```
npm run preview
```

Python Service Deployment

1. **Create virtual environment:**

```
python -m venv .venv
source .venv/bin/activate  # On Windows: .venv\Scripts\activate
```

2. **Install dependencies:**

```
pip install -r requirements.txt
```

3. **Run the service:**

```
uvicorn serviceFast.main:app --reload --host 0.0.0.0 --port 8000
```

_

Troubleshooting

Common Issues

1. **Backend fails to connect to Python service**

• Check Python service is running on port 8000

• Verify `python.api.url` in application.properties

• Check firewall settings

2. **Frontend cannot connect to Backend**

• Verify Backend is running on port 8080

• Check proxy configuration in vite.config.ts

• Ensure CORS is configured in Backend

3. **Model training fails**

• Check CSV file format matches expected schema

• Verify required columns are present

• Check Python service logs for errors

• Ensure sufficient memory for training

4. **LSTM training is very slow**

• Reduce batch size

• Decrease number of epochs

• Use GPU if available

• Reduce model complexity

5. **Database connection errors**

• Verify database is running

• Check connection string in application.properties

• Ensure database credentials are correct

• Create database if it doesn't exist

_

Performance Optimization

Backend Optimization

1. **Database Connection Pooling**

• Configure HikariCP for optimal connection management

• Set appropriate pool size based on load

2. **Caching**

• Enable Spring Cache for frequently accessed data

• Cache dashboard metrics to reduce database queries

3. **Async Processing**

• Use @Async for long-running operations

• Implement background job processing for training

Frontend Optimization

1. **Code Splitting**

• Implement lazy loading for routes

• Split large components into smaller chunks

2. **Chart Optimization**

• Limit data points in charts

• Implement data pagination

• Use chart.js performance optimizations

3. **API Optimization**

• Implement request debouncing

• Cache API responses

• Use WebSocket for real-time updates

Python Service Optimization

1. **Model Optimization**

• Use model quantization for smaller file sizes

• Implement model pruning to reduce size

• Use ONNX format for faster inference

2. **Data Processing**

• Use parallel processing for data preprocessing

• Implement chunked processing for large datasets

• Use efficient data structures (Polars vs Pandas)

3. **API Performance**

• Implement response compression

• Use async endpoints for I/O operations

• Implement request queuing for high load

_

Security Considerations

Authentication & Authorization

1. **JWT Token Authentication**

• Tokens stored securely in localStorage

• Token expiration handling

• Refresh token implementation

2. **Role-Based Access Control**

• Admin, Manager, User roles

• Endpoint-level authorization

• Frontend route guards

Data Security

1. **Input Validation**

• CSV file validation

• SQL injection prevention

• XSS protection

2. **API Security**

• Rate limiting

• CORS configuration

• Request size limits

3. **Model Security**

• Model file encryption

• Secure model storage

• Access control for model endpoints

_

Future Enhancements

Planned Features

1. **Advanced Analytics**

• Anomaly detection

• Trend analysis

• Seasonal decomposition

2. **Model Improvements**

• Additional ML algorithms (Prophet, ARIMA)

• Ensemble methods

• AutoML integration

3. **User Experience**

• Real-time collaboration

• Advanced visualization options

• Mobile application

4. **Infrastructure**

• Kubernetes deployment

• Microservices architecture

• Cloud-native implementation

_

Conclusion

This Workforce Forecasting System represents a comprehensive machine learning application that integrates modern web technologies with advanced predictive analytics. The system provides accurate workforce demand predictions through multiple ML algorithms, real-time monitoring capabilities, and an intuitive user interface.

The modular architecture allows for easy extension and maintenance, while the comprehensive API documentation ensures smooth integration with external systems. The combination of Java Spring Boot, Vue.js, and Python ML services provides a robust foundation for workforce planning and optimization.

_

Document Version: 1.0

Last Updated: August 2026

Author: Workforce Forecasting Team

Workforce Forecasting System

Complete Code Documentation

A comprehensive guide to the workforce forecasting application

Workforce Forecasting System - Complete Code Documentation

Table of Contents

_

Project Overview

The Workforce Forecasting System is a comprehensive machine learning application designed to predict workforce demand across different departments. The system consists of three main components:

• **Backend**: Java Spring Boot application for data management and API services

• **Frontend**: Vue.js application for user interface and visualization

• **Python Service**: Machine learning models for training and prediction

Technology Stack

Backend:

• Java 17+

• Spring Boot

• Spring Data JPA

• PostgreSQL/MySQL database

• RESTful APIs

Frontend:

• Vue.js 3

• TypeScript

• PrimeVue components

• Chart.js for visualizations

• Axios for HTTP requests

• Tailwind CSS

Python ML Service:

• FastAPI

• TensorFlow/Keras

• scikit-learn

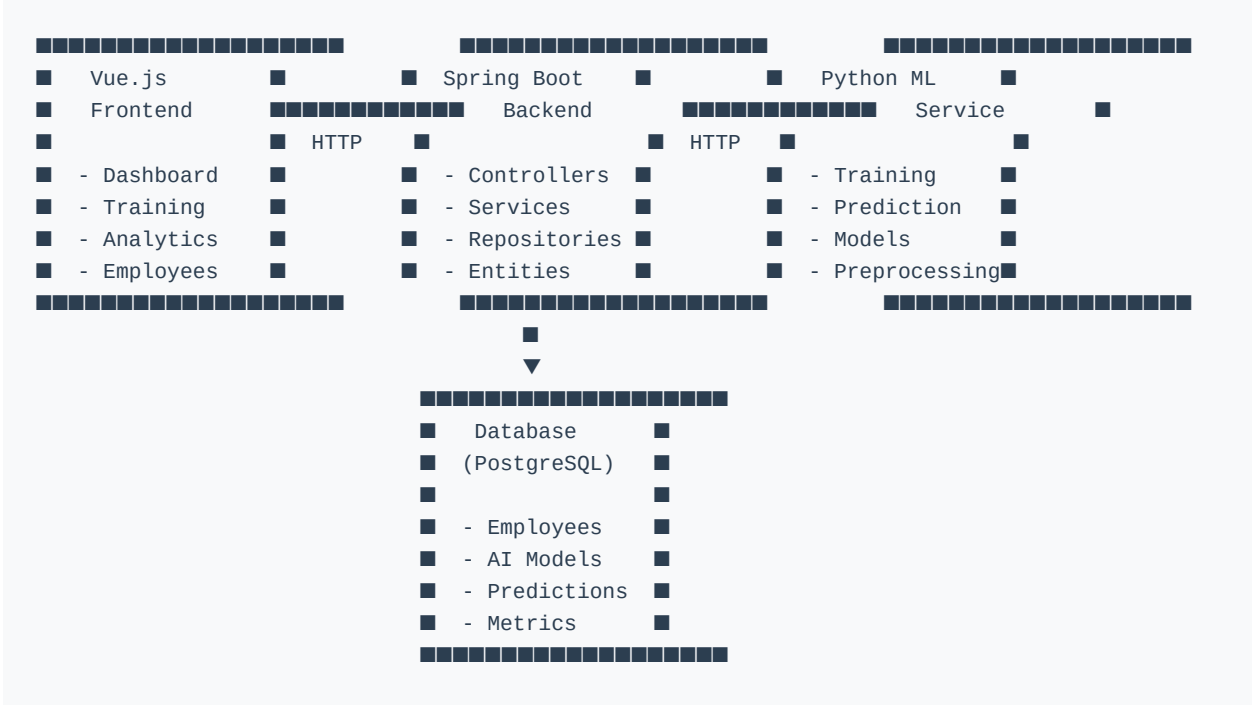
• XGBoost

• Pandas

• NumPy

_

System Architecture



_

Backend Implementation (Java Spring Boot)

Main Application Class

File: `WorkforceApplication.java`

```
package com.boostphysioclinic.workforceapplication;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class WorkForceApplication {
    public static void main(String[] args) {
        SpringApplication.run(WorkForceApplication.class, args);
    }
}
```

Explanation:

• Main entry point for the Spring Boot application

• `@SpringBootApplication` annotation enables auto-configuration and component scanning

• Starts the embedded web server (Tomcat) and initializes the Spring context

Controllers

DashboardController

File: `DashboardController.java`

Purpose: Provides dashboard metrics and visualization data for the frontend.

Key Endpoints:

1. **GET** `/api/dashboard`

• Returns comprehensive dashboard data including metrics and charts

• Fetches latest trained model information from database

• Calculates real-time metrics from prediction data

• Returns line chart data (historical vs predicted demand)

• Returns bar chart data (department performance)

2. **GET** `/api/dashboard/predictions`

• Returns prediction records from CSV files

• Used for populating prediction tables

3. **GET** `/api/dashboard/overview`

• Alias for the main dashboard endpoint

Key Functionality:

```
@GetMapping
public ResponseEntity<Map<String, Object>> getDashboard() {
    // Fetch latest trained model
    Optional<AIModel> latestModel = aiModelRepository.findFirstByOrderByLastTrainedDesc();

    // Get predictions
    List<PredictionRecord> predictions = predictionCsvService.getPredictions();

    // Calculate metrics
    double totalPredicted = predictions.stream()
        .mapToDouble(PredictionRecord::getPredictedDemand)
        .sum();

    // Prepare chart data
    Map<String, Object> lineChartData = new HashMap<>();
    lineChartData.put("labels", labels);
    lineChartData.put("historical", historicalData);
    lineChartData.put("predicted", predictedData);

    return ResponseEntity.ok(response);
}
```

TrainingController

File: `TrainingController.java`

Purpose: Handles model training requests and manages training results.

Key Endpoints:

1. **POST** `/api/train`

• Accepts CSV file and selected algorithms

• Forwards training request to Python ML service

• Saves training results to database

• Returns training response with metrics

2. **GET** `/api/train/cleaned-dataset`

• Downloads the preprocessed dataset used for training

3. **GET** `/api/train/latest-model`

• Returns the most recently trained model information

4. **GET** `/api/train/model-comparisons`

• Returns comparison data for all trained models

Key Functionality:

```
@PostMapping
public ResponseEntity<String> trainModel(
    @RequestParam("file") MultipartFile file,
    @RequestParam("algorithms") List<String> algorithms
) throws Exception {
    return ResponseEntity.ok(trainingService.train(file, algorithms));
}
```

Services

TrainingService

File: `TrainingService.java`

Purpose: Orchestrates model training by communicating with Python ML service.

Key Methods:

1. `train(MultipartFile file, List algorithms)`

• Converts uploaded file to ByteArrayResource

• Constructs multipart request with file and algorithms

• Sends POST request to Python training endpoint

• Parses response and saves results to database

• Returns training response to frontend

2. `saveTrainingResults(String responseBody, String fileName, List algorithms)`

• Parses JSON response from Python service

• Extracts best model and metrics

• Saves AIModel entity with performance metrics

• Saves ModelComparison entities for all trained models

Key Code:

```
public String train(MultipartFile file, List<String> algorithms) throws Exception {
    RestTemplate restTemplate = new RestTemplate();

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.MULTIPART_FORM_DATA);

    ByteArrayResource resource = new ByteArrayResource(file.getBytes()) {
        @Override
        public String getFilename() {
            return file.getOriginalFilename();
        }
    };

    MultiValueMap<String, Object> body = new LinkedMultiValueMap<>();
    body.add("file", resource);
    body.add("algorithms", String.join(",", algorithms));

    ResponseEntity<String> response = restTemplate.postForEntity(
        pythonUrl, request, String.class
    );

    saveTrainingResults(response.getBody(), file.getOriginalFilename(), algorithms);
    return response.getBody();
}
```

Repositories

The backend uses Spring Data JPA repositories for database operations:

• **AIModelRepository**: Manages trained AI model metadata

• **ModelComparisonRepository**: Stores comparison results between models

• **EmployeeRepository**: Manages employee data

• **PredictionResultRepository**: Stores prediction results

• **DatasetRepository**: Manages training datasets

Entities

AIModel

Purpose: Represents a trained machine learning model with its performance metrics.

Key Fields:

• `name`: Model name

• `algorithm`: Algorithm type (Linear Regression, Random Forest, XGBoost, LSTM)

• `version`: Model version

• `status`: Training status (TRAINED, TRAINING, FAILED)

• `rmse`: Root Mean Square Error

• `mae`: Mean Absolute Error

• `mape`: Mean Absolute Percentage Error

• `rSquared`: R-squared score

• `lastTrained`: Timestamp of last training

ModelComparison

Purpose: Stores comparison metrics for different models trained in the same session.

Key Fields:

• `modelName`: Name of the model

• `algorithm`: Algorithm type

• `rmse`, `mae`, `mape`, `rSquared`: Performance metrics

• `status`: Model status (Best, Good)

• `createdAt`: Creation timestamp

_

Frontend Implementation (Vue.js)

Main Application Setup

File: `main.ts`

Purpose: Application entry point that configures Vue, plugins, and global components.

Key Configurations:

1. **PrimeVue Setup**

```
app.use(PrimeVue, {
  theme: {
    preset: Aura,
    options: {
      darkModeSelector: '.dark'
    }
  }
})
```

2. **Global Components Registration**

• Chart, Panel, Card, Button, Tag, Divider

• DataTable, Column, InputText, Select

• FileUpload, Toast, Dropdown, InputNumber

3. **Chart.js Registration**

```
ChartJS.register(
  LineController, LineElement, BarElement, BarController,
  CategoryScale, LinearScale, PointElement, RadialLinearScale,
  Filler, Tooltip, Legend
)
```

App Component

File: `App.vue`

Purpose: Main application layout with sidebar and top menu.

Key Features:

• Conditional rendering of sidebar and top bar based on route

• Responsive layout with flexbox

• Scrollable content area with custom scrollbar styling

• Toast notification integration

Layout Structure:

```
<template>
  <div class="app-wrapper">
    <Toast />
    <topMenuBarComponent v-if="showTopBar" />
    <div class="layout">
      <div class="sideBar" v-if="showSidebar">
        <sideBarComponent />
      </div>
      <div class="content">
        <router-view />
      </div>
    </div>
  </div>
</template>
```

API Client

File: `apiClient.ts`

Purpose: Configured Axios instance for HTTP requests with authentication.

Key Features:

1. **Base Configuration**

```
const api = axios.create({
  baseURL: '/api',
  timeout: 1200000, // 20 minutes for long operations
})
```

2. **JWT Token Interceptor**

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})
```

3. **Error Handling**

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token')
      window.location.href = '/login'
    }
    return Promise.reject(error)
  }
)
```

Dashboard Component

File: `dashboardComponent.vue`

Purpose: Main dashboard displaying workforce metrics, charts, and operational alerts.

Key Features:

1. **KPI Cards**

• Active Workforce

• Forecast Accuracy

• Shift Utilization

• Total Predictions

• Average Demand

• Model Status

• Model Name

• R^2 Score

• RMSE

2. **Line Chart**

• Historical vs Predicted demand over time

• Uses Chart.js with custom styling

• Auto-refreshes every 30 seconds

3. **Bar Chart**

• Department performance comparison

• Horizontal bar chart with target lines

• Color-coded performance indicators

4. **Operational Alerts**

• Real-time alerts with severity levels

• Critical, Warning, Success categories

• Time-stamped alert notifications

5. **Staffing Heatmap**

• Department-wise shift allocation

• Morning, Afternoon, Night shifts

• Total staffing per department

Key Methods:

```
const fetchDashboardMetrics = async () => {
  loading.value = true
  try {
    const response = await CLDashboardService.getDashboardData()

    if (response?.metrics) {
      const backendMetrics = response.metrics as Record<string, string>
      // Update KPI cards with backend data
      metrics.value = [
        {
          title: 'Active Workforce',
          value: backendMetrics['Average Demand'] || 'N/A',
          // ... other metrics
        }
      ]
    }

    // Update chart data
    if (response?.charts) {
      const charts = response.charts as Record<string, any>
      chartData.value = setChartData(
        charts.lineChart.labels,
        charts.lineChart.historical,
        charts.lineChart.predicted
      )
    }
  } catch (error) {
    console.error('Failed to fetch dashboard metrics:', error)
  } finally {
    loading.value = false
```

```
  }
}
```

Auto-refresh Mechanism:

```
onMounted(() => {
  fetchDashboardMetrics()
  // Set up real-time refresh every 30 seconds
  refreshInterval = window.setInterval(() => {
    fetchDashboardMetrics()
  }, 30000)
})
```

_

Python ML/Forecasting Service

Main Application

File: `serviceFast/main.py`

Purpose: FastAPI application entry point that registers API routers.

Code:

```
from fastapi import FastAPI
from serviceFast.controller.train_controller import router as train_router
from serviceFast.controller.prediction_controller.prediction_router import router as prediction_router

app = FastAPI()
app.include_router(train_router)
app.include_router(prediction_router)
```

Training Pipeline

File: `serviceFast/controller/train_pipeline.py`

Purpose: Orchestrates the complete training workflow from data loading to model saving.

Workflow:

1. Load and preprocess dataset
2. Split features and target
3. Train selected models
4. Evaluate models
5. Save best model and results
6. Return training metrics

Model Training

File: `training/model_trainer.py`

Purpose: Coordinates training of multiple machine learning models.

Key Method:

```
class ModelTrainer:
    def train_models(self, selected_models, X_train, X_test, y_train, y_test, lstm_df):
        results = []
        trained_models = {}

        # Linear Regression
        if "Linear Regression" in selected_models:
            lr_model = train_linear_regression(X_train, y_train)
            predictions = lr_model.predict(X_test)
            metrics = evaluate_model(y_test, predictions, "Linear Regression")
            results.append(metrics)
            trained_models["Linear Regression"] = {
                "type": "ml",
                "model": lr_model
            }

        # Random Forest
        if "Random Forest" in selected_models:
            rf_model = train_random_forest(X_train, y_train)
            predictions = rf_model.predict(X_test)
            metrics = evaluate_model(y_test, predictions, "Random Forest")
            results.append(metrics)
            trained_models["Random Forest"] = {
                "type": "ml",
                "model": rf_model
            }

        # XGBoost
        if "XGBoost" in selected_models:
            xgb_model = train_xgboost(X_train, y_train)
```

```
            predictions = xgb_model.predict(X_test)
            metrics = evaluate_model(y_test, predictions, "XGBoost")
            results.append(metrics)
            trained_models["XGBoost"] = {
                "type": "ml",
                "model": xgb_model
            }

        # LSTM (Deep Learning)
        if "LSTM" in selected_models:
            X_train_lstm, X_test_lstm, y_train_lstm, y_test_lstm, X_dashboard, target_scaler, test_departmen
            lstm_model, history = train_lstm(X_train_lstm, y_train_lstm)
            predictions = lstm_model.predict(X_test_lstm, verbose=0)
            predictions = target_scaler.inverse_transform(predictions)
            y_actual = target_scaler.inverse_transform(y_test_lstm.reshape(-1, 1))
            metrics = evaluate_model(y_actual.flatten(), predictions.flatten(), "LSTM")
            results.append(metrics)
            trained_models["LSTM"] = {
                "type": "lstm",
                "model": lstm_model,
                "dashboard_data": X_dashboard,
                "target_scaler": target_scaler
            }

        return results, trained_models, X_dashboard, target_scaler
```

LSTM Model

File: `models/lstm_model.py`

Purpose: Implements Long Short-Term Memory neural network for time series forecasting.

Architecture:

```
model = Sequential([
    Input(shape=(X_train.shape[1], X_train.shape[2])),

    # LSTM Block 1
    LSTM(128, return_sequences=True),
    BatchNormalization(),
    Dropout(0.30),

    # LSTM Block 2
    LSTM(64, return_sequences=False),
    BatchNormalization(),
    Dropout(0.30),

    # Dense Layers
    Dense(32, activation="relu"),
    Dropout(0.20),
    Dense(16, activation="relu"),
    Dense(1)
])
```

Training Configuration:

• Optimizer: Adam (learning rate=0.001)

• Loss: Mean Squared Error (MSE)

• Metrics: Root Mean Square Error (RMSE)

• Epochs: 150

• Batch Size: 64

• Validation Split: 20%

• Callbacks: Early Stopping, Learning Rate Reduction

Callbacks:

```
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=12,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=5,
    min_lr=1e-5,
    verbose=1
)
```

Random Forest Model

File: `models/random_forest_model.py`

Purpose: Implements Random Forest regressor with hyperparameter tuning.

Hyperparameter Grid:

```
param_grid = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [5, 10, 15, 20, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2"]
}
```

Optimization:

• Uses RandomizedSearchCV for efficient hyperparameter search

• 10 iterations with 3-fold cross-validation

• Scoring: Negative Root Mean Square Error

• Parallel processing with n_jobs=-1

Data Preprocessing

File: `preprocessing/preprocessing.py`

Purpose: Prepares raw workforce data for machine learning models.

Key Steps:

1. **Data Loading**

```
def load_dataset(path):
    df = pd.read_csv(path)
    return df
```

2. **Data Cleaning**

```
def preprocess_dataset(df):
    # Remove duplicates
    df.drop_duplicates(inplace=True)

    # Convert date
    df["AttendanceDate"] = pd.to_datetime(df["AttendanceDate"])

    # Sort by department and date
    df = df.sort_values(["Department", "AttendanceDate"])
```

3. **Feature Engineering**

```
# Calendar Features
df["Year"] = df["AttendanceDate"].dt.year
df["Quarter"] = df["AttendanceDate"].dt.quarter
df["Month"] = df["AttendanceDate"].dt.month
df["WeekOfYear"] = df["AttendanceDate"].dt.isocalendar().week.astype(int)
df["DayOfWeek"] = df["AttendanceDate"].dt.dayofweek
df["Weekend"] = (df["DayOfWeek"] >= 5).astype(int)
df["IsMonthStart"] = df["AttendanceDate"].dt.is_month_start.astype(int)
df["IsMonthEnd"] = df["AttendanceDate"].dt.is_month_end.astype(int)

# Historical Features
dept = df.groupby("Department")
df["PreviousDayDemand"] = dept["WorkforceDemand"].shift(1)
df["Previous3DayAverage"] = dept["WorkforceDemand"].transform(
    lambda x: x.shift(1).rolling(3, min_periods=1).mean()
)
df["Previous7DayAverage"] = dept["WorkforceDemand"].transform(
    lambda x: x.shift(1).rolling(7, min_periods=1).mean()
)

# Forecast Target
df["TargetDemand"] = dept["WorkforceDemand"].shift(-1)
```

4. **Data Encoding**

```
def encode_dataset(df):
    categorical_columns = df.select_dtypes(include=["object"]).columns.tolist()
    if "AttendanceDate" in categorical_columns:
        categorical_columns.remove("AttendanceDate")

    if len(categorical_columns) > 0:
        df = pd.get_dummies(df, columns=categorical_columns, drop_first=True)
    return df
```

5. **Train-Test Split**

```
def split_train_test(X, y):
    split_index = int(len(X) * 0.80)
    X_train = X.iloc[:split_index].copy()
    X_test = X.iloc[split_index:].copy()
    y_train = y.iloc[:split_index].copy()
    y_test = y.iloc[split_index:].copy()
    return X_train, X_test, y_train, y_test
```

Model Evaluation

File: `evaluation/evaluation.py`

Purpose: Evaluates model performance using multiple metrics.

Metrics Calculated:

```
def evaluate_model(y_true, predictions, model_name):
    # Root Mean Square Error
    rmse = np.sqrt(mean_squared_error(y_true, predictions))

    # Mean Absolute Error
    mae = mean_absolute_error(y_true, predictions)

    # Mean Absolute Percentage Error
    epsilon = 1e-8
    mape = np.mean(np.abs((y_true - predictions) / (y_true + epsilon))) * 100

    # R-squared Score
    r2 = r2_score(y_true, predictions)

    return {
        "Model": model_name,
        "RMSE": round(rmse, 4),
        "MAE": round(mae, 4),
        "MAPE": round(mape, 2),
        "R2": round(r2, 4)
    }
```

Prediction Service

File: `serviceFast/model_service/prediction_service.py`

Purpose: Handles prediction requests using trained models.

Key Functionality:

```
class PredictionService:
    def __init__(self):
        # Load trained model
        self.model = joblib.load(model_path)

        # Load training feature columns
        with open(feature_path, "r") as file:
            self.training_columns = json.load(file)

        # Load model info
        with open(info_path, "r") as file:
            self.model_info = json.load(file)

    def predict(self, dataframe):
        # Add missing columns with zeros
        missing_columns = [col for col in self.training_columns if col not in dataframe.columns]
        if missing_columns:
            missing_df = pd.DataFrame(0, index=dataframe.index, columns=missing_columns)
            dataframe = pd.concat([dataframe, missing_df], axis=1)

        # Keep only training columns
        dataframe = dataframe[self.training_columns]

        # Make predictions
        predictions = self.model.predict(dataframe)

        # Format results
        results = []
        for i, pred in enumerate(predictions):
            result = {
```

```
                "attendanceDate": dataframe.iloc[i].get("AttendanceDate", ""),
                "department": dataframe.iloc[i].get("Department", ""),
                "actualDemand": dataframe.iloc[i].get("WorkforceDemand", None),
                "predictedDemand": float(pred)
            }
            results.append(result)

        return {
            "model": self.model_info["Model"],
            "total_records": len(predictions),
            "predictions": predictions.tolist(),
            "results": results
        }
```

Prediction Controller

File:
`serviceFast/controller/prediction_controller/prediction_router.py`

Purpose: API endpoint for handling prediction requests.

Endpoint:

```
@router.post("")
async def predict(file: UploadFile = File(...)):
    try:
        # Read uploaded CSV
        dataframe = pd.read_csv(file.file)

        # Apply preprocessing
        prediction_df = prepare_prediction_data(dataframe)

        # Predict
        result = prediction_service.predict(prediction_df)

        return result
    except Exception:
        traceback.print_exc()
        raise HTTPException(
            status_code=500,
            detail="Prediction failed. Check server logs."
        )
```

_

Data Flow and Integration

Training Workflow

1. **User uploads CSV file** via Frontend UI

2. **Frontend sends POST request** to `/api/train` with file and selected algorithms

3. **Backend TrainingService** forwards request to Python ML service

4. **Python service** processes the file:

• Loads and preprocesses data

• Trains selected models

• Evaluates performance

• Saves best model

5. **Python returns** training results with metrics

6. **Backend saves** results to database (AIModel, ModelComparison tables)

7. **Backend returns** response to Frontend

8. **Frontend updates** UI with training results

Prediction Workflow

1. **User uploads prediction CSV** via Frontend

2. **Frontend sends POST request** to Python prediction endpoint

3. **Python service**:

• Preprocesses prediction data

• Loads trained model

• Makes predictions

• Returns formatted results

4. **Results displayed** in Frontend dashboard

Dashboard Data Flow

1. **Frontend polls** `/api/dashboard` every 30 seconds

2. **Backend fetches**:

• Latest trained model from database

• Prediction records from CSV files

3. **Backend calculates**:

• Real-time metrics (accuracy, averages)

• Chart data (line chart, bar chart)

4. **Frontend updates**:

• KPI cards

• Charts

• Alerts panel

_

Key Algorithms and Models

Linear Regression

Purpose Baseline model for workforce demand prediction.

Characteristics:

• Simple interpretable model

• Fast training time

• Good for linear relationships

• Limited capture of complex patterns

Random Forest

Purpose: Ensemble learning method for improved accuracy.

Characteristics:

• Multiple decision trees

• Handles non-linear relationships

• Robust to overfitting

• Feature importance analysis

• Moderate training time

XGBoost

Purpose: Gradient boosting framework for high performance.

Characteristics:

• Advanced gradient boosting

• Handles missing values

• Regularization to prevent overfitting

• High accuracy

• Faster training than Random Forest

LSTM (Long Short-Term Memory)

Purpose: Deep learning model for time series forecasting.

Characteristics:

• Captures temporal dependencies

• Handles sequential data

• Complex architecture with multiple layers

• Highest accuracy potential

• Longer training time

• Requires more data

Architecture Details:

• Two LSTM layers (128 and 64 units)

• Batch normalization for stability

• Dropout regularization (30% for LSTM, 20% for Dense)

• Dense layers for feature transformation

• Adam optimizer with learning rate scheduling

_

API Endpoints Reference

Backend API Endpoints

Dashboard Endpoints

- `GET /api/dashboard` - Get dashboard metrics and charts
- `GET /api/dashboard/predictions` - Get prediction records
- `GET /api/dashboard/overview` - Get dashboard overview

Training Endpoints

- `POST /api/train` - Train models with uploaded dataset
- Parameters: `file` (MultipartFile), `algorithms` (List)
- Returns: Training results with metrics
- `GET /api/train/cleaned-dataset` - Download preprocessed dataset
- `GET /api/train/latest-model` - Get latest trained model info
- `GET /api/train/model-comparisons` - Get model comparison data

Employee Endpoints

- `GET /api/employees` - Get all employees
- `POST /api/employees` - Create new employee
- `PUT /api/employees/{id}` - Update employee
- `DELETE /api/employees/{id}` - Delete employee

Analytics Endpoints

- `GET /api/analytics` - Get analytics data
- `GET /api/analytics/performance` - Get performance metrics

Authentication Endpoints

- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `POST /api/auth/register` - User registration

Python ML Service Endpoints

Training Endpoints

- `POST /train` - Train ML models
- Parameters: `file` (UploadFile), `algorithms` (comma-separated string)
- Returns: JSON with best model, metrics, and comparison data

Prediction Endpoints

- `POST /predict` - Make predictions
- Parameters: `file` (UploadFile with prediction data)
- Returns: JSON with predictions and metadata

_

Database Schema

AIModel Table

Column	Type	Description
id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type

Column	Type	Description
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error

id	BIGINT	Primary key
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error

version	VARCHAR	Model version
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

last_trained	TIMESTAMP	Last training timestamp

ModelComparison Table

Column	Type	Description
id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score

id	BIGINT	Primary key
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

model_name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
status	VARCHAR	Model status
created_at	TIMESTAMP	Creation timestamp

algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error

algorithm	VARCHAR	Algorithm type
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

status	VARCHAR	Model status
created_at	TIMESTAMP	Creation timestamp

created_at	TIMESTAMP	Creation timestamp

Employee Table

Column	Type	Description
id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

id	BIGINT	Primary key
name	VARCHAR	Model name
algorithm	VARCHAR	Algorithm type
version	VARCHAR	Model version

id	BIGINT	Primary key
status	VARCHAR	Training status
rmse	DOUBLE	Root Mean Square Error
mae	DOUBLE	Mean Absolute Error
mape	DOUBLE	Mean Absolute Percentage Error
r_squared	DOUBLE	R-squared score
last_trained	TIMESTAMP	Last training timestamp

first_name	VARCHAR	Employee first name
last_name	VARCHAR	Employee last name
email	VARCHAR	Employee email
department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

last_name	VARCHAR	Employee last name
email	VARCHAR	Employee email
department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

email	VARCHAR	Employee email
department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

department	VARCHAR	Department name
position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

position	VARCHAR	Job position
hire_date	DATE	Hire date
status	VARCHAR	Employment status

hire_date	DATE	Hire date
status	VARCHAR	Employment status

status	VARCHAR	Employment status

_

Configuration Files

Backend Configuration

application.properties

```
# Database Configuration
spring.datasource.url=jdbc:postgresql://localhost:5432/workforce_db
spring.datasource.username=postgres
spring.datasource.password=password
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

# Python Service Configuration
python.api.url=http://localhost:8000/train
python.download.url=http://localhost:8000/download/cleaned-dataset

# Server Configuration
server.port=8080
```

Frontend Configuration

vite.config.ts

```
export default defineConfig({
  plugins: [vue()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true
      }
    }
  }
})
```

Python Configuration

requirements.txt

```
fastapi==0.115.0
uvicorn==0.32.0
pydantic==2.9.2
python-multipart==0.0.12
numpy==1.26.4
pandas==2.1.0
scikit-learn
xgboost
tensorflow
joblib
```

_

Deployment Instructions

Backend Deployment

1. **Build the application:**

```
./gradlew build
```

2. **Run the application:**

```
java -jar build/libs/workForceApplication.jar
```

3. **Or run with Gradle:**

```
./gradlew bootRun
```

Frontend Deployment

1. **Install dependencies:**

```
npm install
```

2. **Development server:**

```
npm run dev
```

3. **Production build:**

```
npm run build
```

4. **Preview production build:**

```
npm run preview
```

Python Service Deployment

1. **Create virtual environment:**

```
python -m venv .venv
source .venv/bin/activate  # On Windows: .venv\Scripts\activate
```

2. **Install dependencies:**

```
pip install -r requirements.txt
```

3. **Run the service:**

```
uvicorn serviceFast.main:app --reload --host 0.0.0.0 --port 8000
```

_

Troubleshooting

Common Issues

1. **Backend fails to connect to Python service**

• Check Python service is running on port 8000

• Verify `python.api.url` in application.properties

• Check firewall settings

2. **Frontend cannot connect to Backend**

• Verify Backend is running on port 8080

• Check proxy configuration in vite.config.ts

• Ensure CORS is configured in Backend

3. **Model training fails**

• Check CSV file format matches expected schema

• Verify required columns are present

• Check Python service logs for errors

• Ensure sufficient memory for training

4. **LSTM training is very slow**

• Reduce batch size

• Decrease number of epochs

• Use GPU if available

• Reduce model complexity

5. **Database connection errors**

• Verify database is running

• Check connection string in application.properties

• Ensure database credentials are correct

• Create database if it doesn't exist

_

Performance Optimization

Backend Optimization

1. **Database Connection Pooling**

• Configure HikariCP for optimal connection management

• Set appropriate pool size based on load

2. **Caching**

• Enable Spring Cache for frequently accessed data

• Cache dashboard metrics to reduce database queries

3. **Async Processing**

• Use @Async for long-running operations

• Implement background job processing for training

Frontend Optimization

1. **Code Splitting**

• Implement lazy loading for routes

• Split large components into smaller chunks

2. **Chart Optimization**

• Limit data points in charts

• Implement data pagination

• Use chart.js performance optimizations

3. **API Optimization**

• Implement request debouncing

• Cache API responses

• Use WebSocket for real-time updates

Python Service Optimization

1. **Model Optimization**

• Use model quantization for smaller file sizes

• Implement model pruning to reduce size

• Use ONNX format for faster inference

2. **Data Processing**

• Use parallel processing for data preprocessing

• Implement chunked processing for large datasets

• Use efficient data structures (Polars vs Pandas)

3. **API Performance**

• Implement response compression

• Use async endpoints for I/O operations

• Implement request queuing for high load

_

Security Considerations

Authentication & Authorization

1. **JWT Token Authentication**

• Tokens stored securely in localStorage

• Token expiration handling

• Refresh token implementation

2. **Role-Based Access Control**

• Admin, Manager, User roles

• Endpoint-level authorization

• Frontend route guards

Data Security

1. **Input Validation**

• CSV file validation

• SQL injection prevention

• XSS protection

2. **API Security**

• Rate limiting

• CORS configuration

• Request size limits

3. **Model Security**

• Model file encryption

• Secure model storage

• Access control for model endpoints

_

Future Enhancements

Planned Features

1. **Advanced Analytics**

• Anomaly detection

• Trend analysis

• Seasonal decomposition

2. **Model Improvements**

• Additional ML algorithms (Prophet, ARIMA)

• Ensemble methods

• AutoML integration

3. **User Experience**

• Real-time collaboration

• Advanced visualization options

• Mobile application

4. **Infrastructure**

• Kubernetes deployment

• Microservices architecture

• Cloud-native implementation

_

Conclusion

This Workforce Forecasting System represents a comprehensive machine learning application that integrates modern web technologies with advanced predictive analytics. The system provides accurate workforce demand predictions through multiple ML algorithms, real-time monitoring capabilities, and an intuitive user interface.

The modular architecture allows for easy extension and maintenance, while the comprehensive API documentation ensures smooth integration with external systems. The combination of Java Spring Boot, Vue.js, and Python ML services provides a robust foundation for workforce planning and optimization.

_

Document Version: 1.0

Last Updated: August 2026

Author: Workforce Forecasting Team